

PA1

Sam Scott

AI Disclosure:

Model used: ChatGPT 5.6

I used it when implementing the scrabble path cost and I verified the solution I adapted from it by testing several word pairs in the dogcat game. Here is the exact block of code I adapted from AI:

```
elif self.cost == 'scrabble':  
    scrabble_values = {  
        'a': 1, 'e': 1, 'i': 1, 'o': 1, 'u': 1,  
        'l': 1, 'n': 1, 's': 1, 't': 1, 'r': 1,  
        'd': 2, 'g': 2,  
        'b': 3, 'c': 3, 'm': 3, 'p': 3,  
        'f': 4, 'h': 4, 'v': 4, 'w': 4, 'y': 4,  
        'k': 5,  
        'j': 6, 'x': 6,  
        'q': 10, 'z': 10  
    }  
    position, letter = action  
    return c + scrabble_values[letter]
```

(1) Heuristic Design and Admissibility

Steps:

Pseudocode:

Function steps(state, goal):

 num = 0

 for i = 0 to length(state) - 1:

 if state[i] != goal[i]

 num++

return num

This heuristic counts the number of letters where the current word differs from the goal word. Every incorrect letter must be changed at least once to reach the goal word, so if there are n incorrect letters, at least n legal actions are necessary to reach the goal word. This heuristic only counts the minimum number of actions required to reach the

goal word so it can never overestimate the optimal remaining cost, so the steps heuristic is admissible.

Scrabble:

Pseudocode:

```
function scrabble(state, goal):  
    value = 0  
    for i = 0 to length (state) - 1:  
        if state[i] != goal[i]:  
            estimate += scrabble_values[i] #returns scrabble value  
    return value
```

This heuristic calculates the sum of scrabble values of the goal letters at positions that are currently incorrect. Each action costs the value of the newly inserted letter, so every incorrect letter must eventually be changed to its goal letter and the cost of changing an incorrect letter cannot be avoided. This heuristic does not account for any suboptimal steps that the program may need to get the goal word and those can only increase the cost. Therefore, scrabble can never overestimate the minimum cost and is admissible.

Frequency:

Pseudocode:

```
function frequency(state, goal):  
    num = 0  
    for i = 0 to length(state) - 1  
        if state[i] != goal[i]:  
            num++  
    return num
```

This heuristic counts the number of letters where the current word is incorrect. The cost of entering a new word is $1 + \text{rarity}(w)$, so every action has a cost of at least 1. If the word differs from the goal by n letters, then there must be at least n actions. Since each action costs at least 1, the total cost is at least n . It cannot overestimate the cost because it does not account for suboptimal steps when getting the goal word, so frequency is admissible. It is similar to steps because they both must have a cost of at least n .

(2) Does the heuristic actually help?

WHY → ASK

Cost Measure	UCS Cost	UCS Expanded	A* Cost	A* Expanded	Reduction
--------------	----------	--------------	---------	-------------	-----------

Steps	5	773	5	16	97.93
Scrabble	9	765	9	13	98.30
Frequency	38.8554065 2	808	38.8554065 2	755	6.56

Bear → Duck

Cost Measure	UCS Cost	UCS Expanded	A* Cost	A* Expanded	Reduction
Steps	4	691	4	6	99.13
Scrabble	11	2981	11	6	99.80
Frequency	38.2402284	1084	38.2402284	720	33.58

1. UCS and A* always returned the same optimal cost.
 2. For both word pairs, scrabble produced the largest percentage reduction in expanded nodes.
 3. For both word pairs, frequency was the least informative. For why → ask there was a 6.56% reduction and for bear → duck there was a 33.58% reduction. This suggests that the corresponding heuristic for frequency was the least helpful since it provided the least improvement over uniform-cost search.
 4. **Steps:** this heuristic counts the number of incorrect letters which gives a lower bound on the number of actions needed to reach the goal. It also ignores any suboptimal actions since intermediate words must be in the dictionary.
- Scrabble:** this heuristic sums the scrabble values of the correct letters in incorrect positions. It accounts for the unavoidable costs of changing a letter to the correct one, so it gives more information about the current state than just incorrect letters.
- Frequency:** this heuristic accounts for the minimum number of letter changes and the rarity cost of the goal. It ignores the rarity cost of suboptimal intermediate steps which can make the prediction much smaller than the actual cost.

(3) Reversibility and direction-dependent costs

Steps

For this cost measure, the optimal cost from W1 to W2 is always the same as the optimal cost from W2 to W1. Each action only changes one letter and costs exactly one, and since every legal action is reversible, any path from W1 to W2 can also be followed backwards from W2 to W1. Therefore, the optimal cost of both directions is the same.

Scrabble

For this cost measure, the optimal cost does not necessarily have to be the same in both directions. Even though every action is reversible, the cost depends on the letter being inserted, not the action. For example, dog $\rightarrow$ cog is not the same as cog $\rightarrow$ dog, since c has a value of 3 and d has a value of 2, so dog $\rightarrow$ cog costs 3 while cog $\rightarrow$ dog costs 2.

Frequency

For this cost measure, the optimal cost does not necessarily have to be the same in both directions. The cost of moving into a word is $1 + \text{rarity}(w)$, so the cost of W1 $\rightarrow$ W2 depends entirely on the rarity of W2, while going backwards depends on the rarity of W1. So any two words whose rarities are different would have different costs going backward and forward.

(4) LLM heuristic audit

Model: Gemini 3.6–Thinking

Prompt: I gave it the exact prompt from the assignment

Proposed Heuristic:

The rarity-aware heuristic $h_r(n)$ enhances the baseline Hamming distance $h_0(n)$ by incorporating guaranteed lower-bound word rarity costs into the estimate. 

- **Core Mechanism:** While $h_0(n)$ only counts the minimum number of letter changes (1 per step), $h_r(n)$ accounts for the actual transition costs ($1 + \text{rarity}$) required by the objective. 
- **Goal Rarity $R(g)$:** Added for any non-goal state because the final transition of any valid path must enter the goal state g , incurring $R(g)$. 
- **Minimum Neighbor Rarity $\min R(w)$:** Added when a state is two or more steps away ($h_0(n) \geq 2$) because the first transition must move to a distinct intermediate neighbor, incurring at least the rarity of the cheapest adjacent word. 
- **Admissibility:** Remains valid ($h_r(n) \leq h^*(n)$) because it only adds mandatory costs that every path is guaranteed to pay. 
- **Informativeness:** Provides higher, more accurate lower bounds ($h_r(n) \geq h_0(n)$), helping search algorithms like A* evaluate fewer nodes and reach the optimal solution faster. 

Formula for heuristic:

$$h_r(n) = \begin{cases} 0 & \text{if } n = g \\ h_0(n) + R(g) & \text{if } h_0(n) = 1 \\ h_0(n) + R(g) + \min_{w \in N(n)} R(w) & \text{if } h_0(n) \geq 2 \text{ and } N(n) \neq \emptyset \\ h_0(n) + R(g) & \text{if } h_0(n) \geq 2 \text{ and } N(n) = \emptyset \end{cases}$$

Variable Definitions:

- n : The current word state.
- g : The target goal word state.
- $h_0(n)$: The baseline Hamming distance (number of character positions where state n differs from g).
- $R(w)$: The nonnegative rarity value of word w .
- $N(n)$: The set of valid dictionary words reachable from state n in exactly one legal step.

👉 🔍 ↻ 📄 ...

Proof: Any valid path to the goal word pays a total cost of k + any intermediate steps that may be needed to reach the goal. This heuristic is admissible because it lower-bounds the total cost across three cases. If we are already at the goal, no moves remain so the cost is 0. If we are one step away, every valid path takes $k \geq 1$ so there is at least a cost of 1. If we are two or more steps away, every valid path to the goal word takes $k \geq 2$ steps. The path must pay at least k steps plus the rarity of the cheapest adjacent neighbor, goal rarity.

Test:

Why → Ask

h_0 gives a solution cost of 38.86 and expands 755 nodes.

The AI-generated heuristic gives a solution cost of 38.86 and expands 497 nodes.

Bear → Duck

h_0 gives a solution cost of 38.24 and expands 720 nodes.

The AI-generated heuristic gives a solution cost of 38.24 and expands 201 nodes.

The proposed heuristic produced the same solution as h_0 , but the lower number of expanded nodes from the proposed heuristic compared to h_0 gives evidence that it is more useful than h_0 for these two frequency-cost problems.